Notes: Kafka Architecture - Part 1

Event Driven Architecture

1 Introduction to Kafka

- **What is Kafka?** It is a distributed event streaming platform that allows you to publish, store, and consume events.
- **Streaming Model:** Unlike simple pub/sub models, streaming allows events to be stored so they can be replayed or consumed at a later time.

2 Core Components of Kafka

- **Producer:** An application that publishes events to Kafka. It simply sends out the event and does not care about who or what will consume it.
- **Topic:** Think of a topic as a logical category or "folder" to which events are published. Many producers can write to the same topic.
- **Broker:** A single Kafka server instance. A broker physically stores data (some partitions of some topics) and serves clients.

3 Partitions

A topic is divided into multiple partitions. A partition is a **physical, ordered, and append-only log**.

- **Physical:** Partitions are where the actual events/logs are stored on the disk.
- **Ordered (Offsets):** Events are written sequentially. Whenever an event enters a partition, it is assigned an incrementing ID called an **offset**.
  - Each partition maintains its own independent offset counter.
  - **Crucial Rule:** Order is *only* guaranteed within a single partition. Kafka does not guarantee global ordering across multiple partitions in the same topic.
  - Consumers must read these events sequentially by their offset.
- **Append-Only Log:** New events are strictly appended to the end of the partition file.

4 Segments and Index Files

To prevent a single partition log from becoming too massive and slow to search, Kafka breaks partition logs into smaller **segments**.

- **Segment Naming:** Segments are named based on their starting offset (e.g., a segment named `0000500.log` holds offsets starting from 500).
- **Efficiency:** This allows Kafka to directly jump to the specific file chunk that contains the requested offset range, speeding up memory loading.
- **Index Files:** Instead of scanning a whole segment linearly, Kafka maintains index files. These map specific offsets to their physical byte positions in the memory.
  - To save memory, not every offset is indexed. An index entry is created every N bytes (which is configurable).

5 How Producers Assign Partitions

How does a producer know which partition to send an event to? Kafka uses partitioning algorithms:

1. **Key-Based Partitioning:** The producer provides a key (like an Order ID). Kafka computes a hash of this key modulo the total number of partitions. This ensures all events with the same key always go to the same partition.
2. **Round-Robin:** If no key is provided, Kafka simply distributes the events evenly across available partitions in a sequential loop.
3. **Custom Partitioning:** You can write custom logic (e.g., routing events from India to Partition 0, and US to Partition 1).